

Segurança e Autenticação

Protegendo Senhas em Aplicações Web

Por que e como proteger dados sensíveis dos usuários

! O Problema: Senhas em Texto Plano

NUNCA armazene senhas em texto plano!

```
# ❌ PERIGOSO - NÃO FAÇA ISSO! usuarios = { "joao":  
"senha123", "maria": "abc123" }
```

Por quê?

- Se o banco de dados vazar, **todas as senhas** ficam expostas
- Funcionários com acesso ao DB podem ver as senhas
- Usuários que reutilizam senhas ficam vulneráveis em outros sites
- Violação de privacidade e confiança



Solução: Hash de Senhas

Hash é uma **função unidirecional** que transforma dados em uma string fixa.



Características Importantes:

- **Unidirecional**: impossível reverter o hash para obter a senha
- **Determinístico**: mesma entrada = mesmo hash
- **Efeito avalanche**: pequena mudança = hash completamente diferente

```
# Exemplo com SHA-256 hash("senha123") →  
ef92b778bafe771e89... hash("senha124") →  
9a3d6f2c1b8e4d7a32...
```

Rainbow Table Attack

Uma **rainbow table** é um banco de dados pré-computado de hashes comuns.

Como funciona o ataque:

1. Atacante cria uma tabela gigante: senha → hash
2. Pré-computa milhões/bilhões de senhas comuns
3. Quando obtém um hash vazado, busca na tabela
4. Se encontrar, descobre a senha original instantaneamente

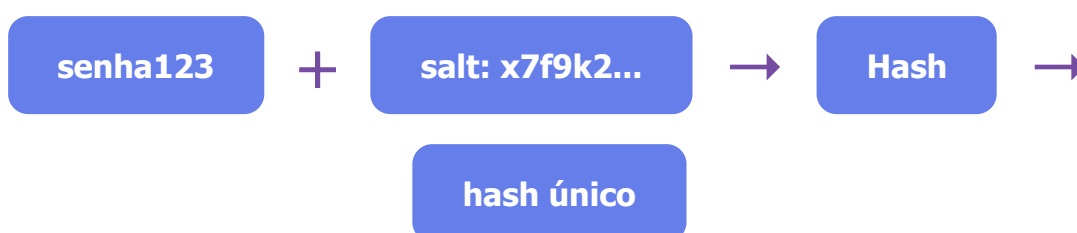
```
# Rainbow Table (exemplo simplificado) "123456" →  
8d969eef6ecad3c29a3a... "password" →  
5e884898da28047151d0... "senha123" →  
ef92b778bafe771e89a2... "admin" →  
8c6976e5b5410415bde9...
```

Problema: Senhas comuns são quebradas instantaneamente!



A Solução: Salt (Sal)

Um **salt** é um valor aleatório único adicionado a cada senha antes do hash.



Benefícios:

- **Rainbow tables inúteis**: cada senha tem salt único
- **Hashes diferentes**: mesma senha = hashes diferentes para usuários diferentes
- **Proteção extra**: atacante precisa quebrar cada hash individualmente

```
# Com Salt user1: hash("senha123" + "abc123xyz") → 7f2a9b... user2: hash("senha123" + "xyz789abc") → 3d8f1c... # Mesma senha, hashes completamente diferentes!
```



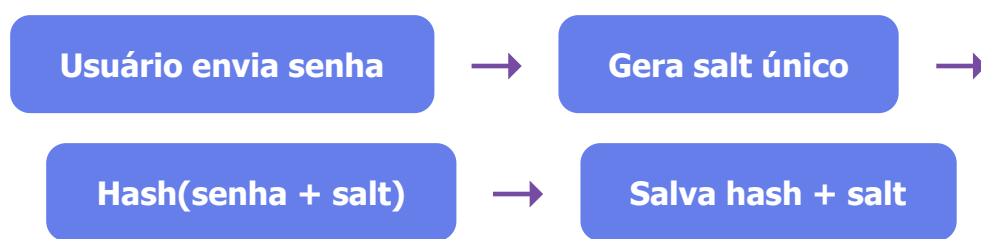
Implementação Prática

```
import hashlib import os # 1. Gerar salt aleatório  
(32 bytes) salt = os.urandom(32).hex() # 2. Combinar  
senha + salt senha_com_salt = "senha123" + salt # 3.  
Criar hash hash_final = hashlib.sha256(  
senha_com_salt.encode('utf-8') ).hexdigest() # 4.  
Armazenar AMBOS: hash E salt no DB # user = { #  
"hash": hash_final, # "salt": salt # }
```

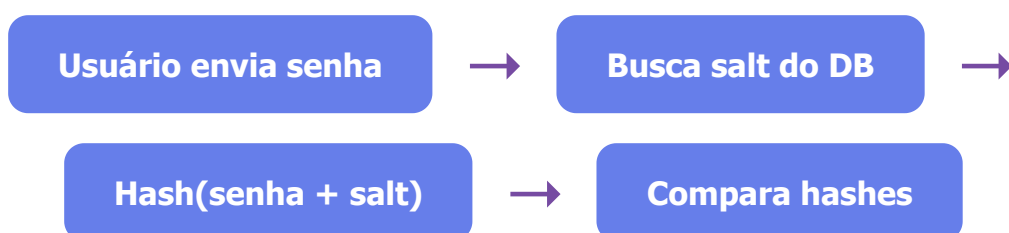
⚠ O salt **não é secreto** - pode ser armazenado junto com o hash!

Fluxo de Autenticação Moderna

1 Registro (Sign Up):



2 Login (Sign In):



✓ **Hashes iguais** = Senha correta = Login permitido

✗ **Hashes diferentes** = Senha incorreta = Login negado

✓ Boas Práticas Modernas

✗ NÃO Use:

- MD5
- SHA-1
- SHA-256 simples

Muito rápidos = vulneráveis a brute force

✓ Use:

- **bcrypt**
- **Argon2**
- **PBKDF2**

Lentos propositalmente + salt automático

Por que algoritmos específicos para senhas?

- **Lentidão intencional** : dificulta ataques de força bruta
- **Cost factor ajustável** : pode aumentar a dificuldade com o tempo
- **Salt automático** : já incluído no algoritmo
- **Resistência a hardware especializado** : protege contra GPUs/ASICs



Resumo

1. Nunca armazene senhas em texto plano

Use hash para proteger as senhas no banco de dados

2. Sempre use salt

Salt único por usuário previne rainbow table attacks

3. Use algoritmos apropriados

bcrypt, Argon2 ou PBKDF2 em produção (não SHA-256)

4. Autenticação = Comparação de hashes

$\text{Hash}(\text{senha_digitada} + \text{salt}) == \text{hash_armazenado}$



Segurança começa com boas práticas!